

ASIGNATURA PROYECTO 2

ENTREGA 1 - DEFINICIÓN DE PRODUCTO

ESTUDIANTES:

**JUAN MANUEL ANTE COLINA
ASHLEE VANESSA CAMPAZ VALENCIA
MÓNICA ALEJANDRA CASTELLANOS MÉNDEZ
LINA ALEXANDRA DIAZ OSPINA
ANA ISABEL QUIRA FERNÁNDEZ
ANDREA CAROLINA REALPE MUÑOZ**

FECHA: 05 de Abril de 2026

UNIVERSIDAD DEL CAUCA POPAYÁN 2026

Introducción

El presente documento corresponde a la segunda entrega del proyecto "Caravana del Retorno, Florencia", en el cual se consolida la documentación técnica y de proceso correspondiente al Sprint 2, abarcando la arquitectura del sistema, los artefactos de diseño, las historias de usuario refinadas y el reporte de la reunión retrospectiva del equipo. El sistema tiene como propósito centralizar la gestión de retornantes, colonias, eventos, multimedia y reportes para la comunidad organizada en torno al Retorno Florencia, respondiendo a los requerimientos identificados en conjunto con los stakeholders del proyecto.

Para mayor detalle sobre las decisiones de diseño, especificaciones técnicas e historial de cambios, se puede consultar la wiki del proyecto en el siguiente enlace: [Wiki del proyecto](#).

Arquitectura del sistema

La arquitectura propuesta para el sistema “Caravana del retorno, Florencia” es una arquitectura monolítica modular, en base a las reuniones con los clientes se identificaron diferentes conjuntos de funcionalidades estrechamente relacionadas entre sí, señalando una clara división lógica, en módulos como: usuarios, colonias, eventos, anuncios, multimedia, donaciones y reportes, además el aplicativo está dirigido a una población específica de usuarios, según el conocimiento de los expertos en el negocio el número máximo de interacciones concurrentes puede alcanzar los 500 usuarios, cantidad que no supone una la necesidad de escalado independiente, por lo que en este contexto no es necesario asumir la sobrecarga operativa que implica una arquitectura distribuida como microservicios. Como lo menciona Newman, los microservicios introducen retos adicionales de despliegue, monitoreo y consistencia distribuida que solo se justifican cuando existen necesidades claras de escalado independiente [1].

En términos de despliegue, un único artefacto facilita la integración continua, reduce costos de infraestructura y simplifica la gestión de seguridad de datos personales, aspecto crítico en el sistema. Además, el requerimiento de hasta 500 usuarios concurrentes puede manejarse eficazmente mediante escalado vertical u horizontal del monolito, sin necesidad de fragmentar la aplicación.

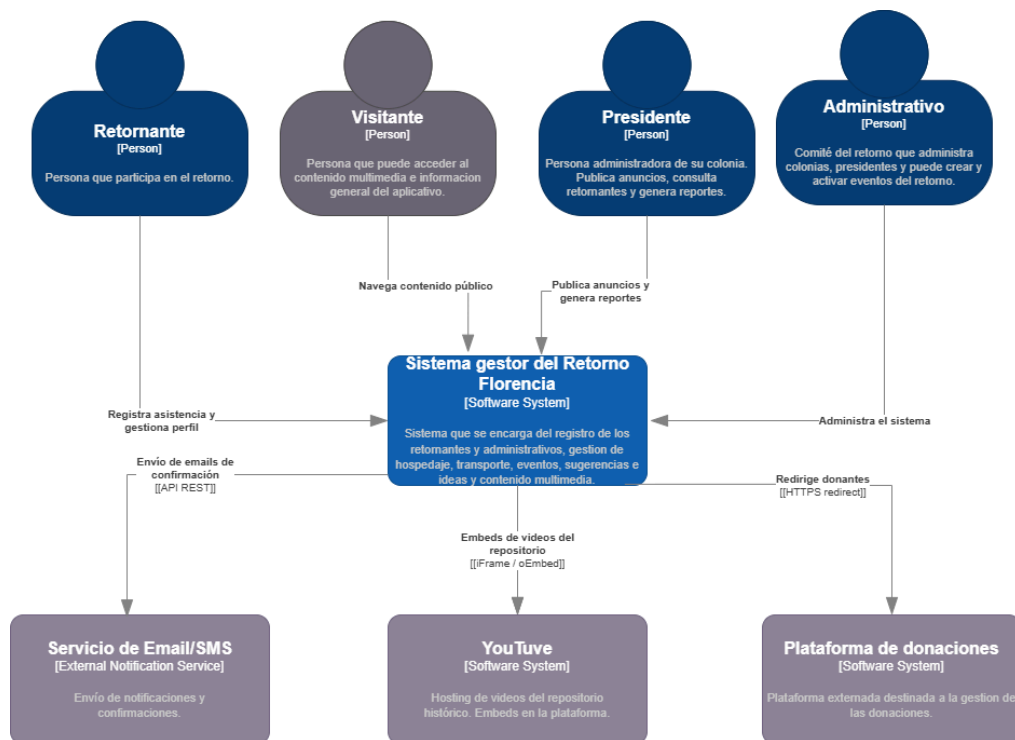


Figura 2. Diagrama de contexto c4

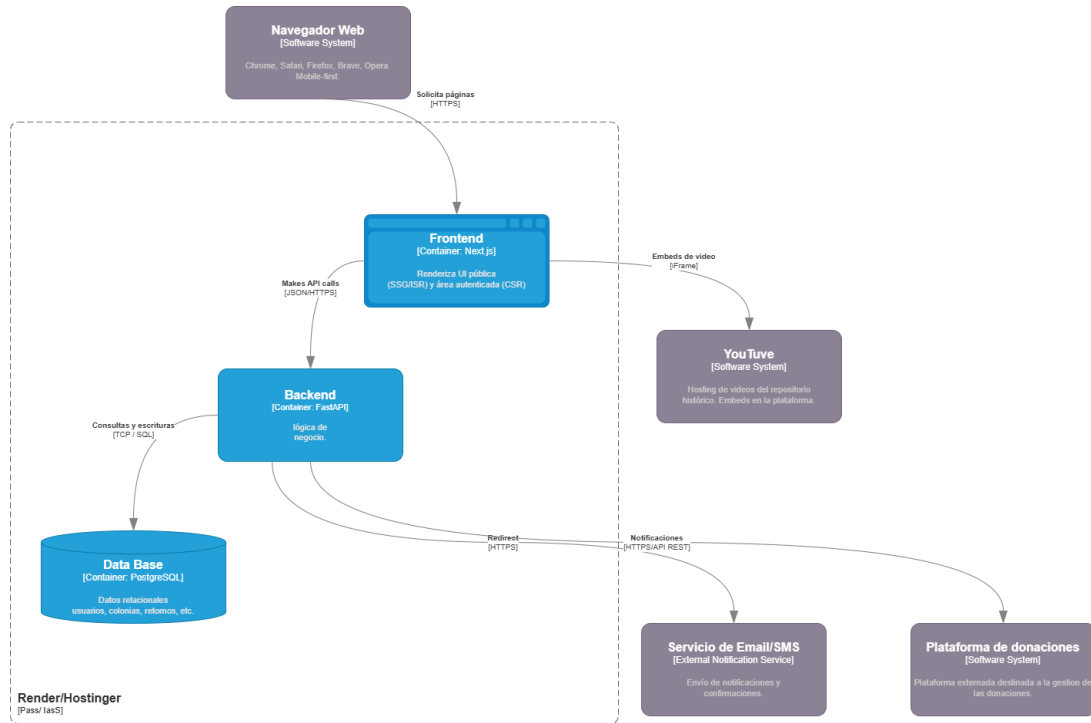


Figura 3. Diagrama de contenedores C4

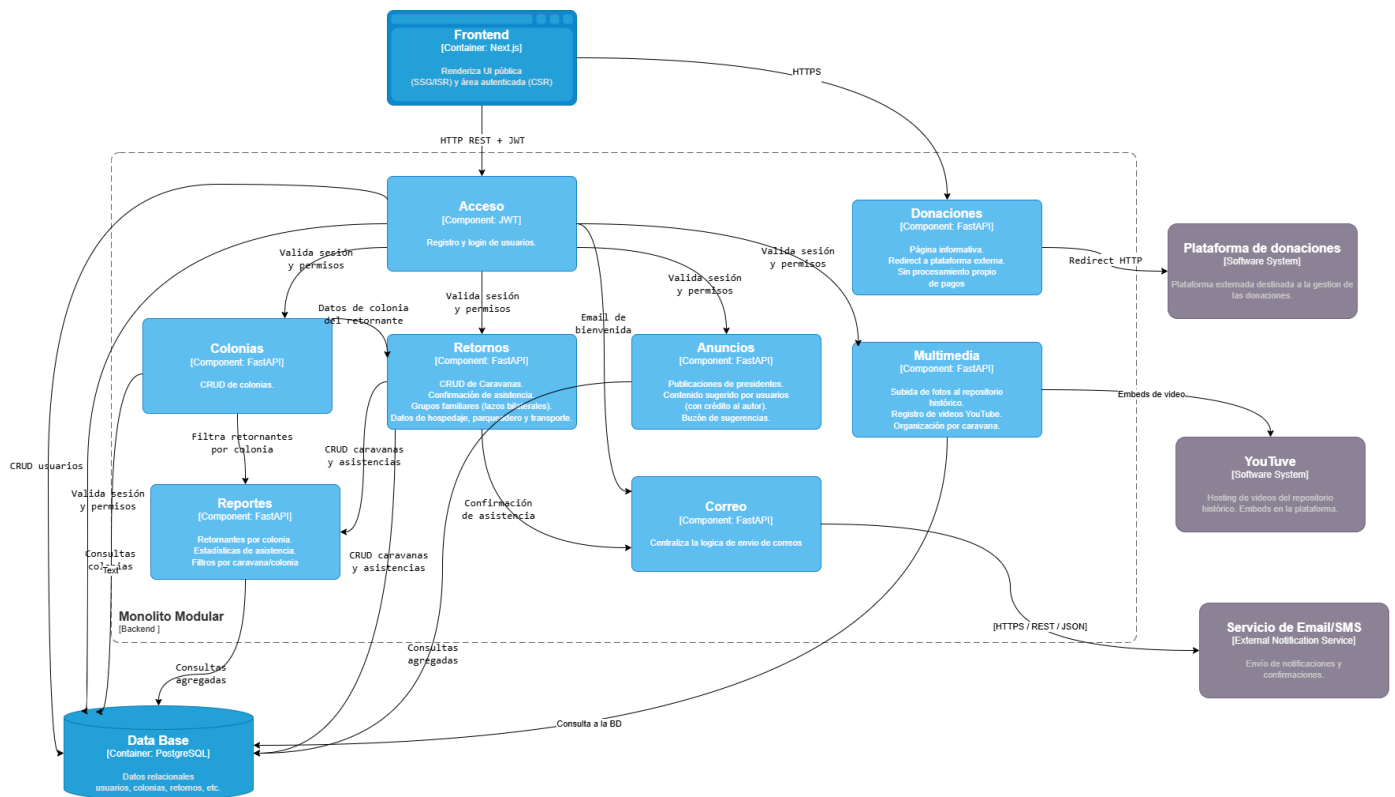


Figura 4. Diagramas de componentes c4

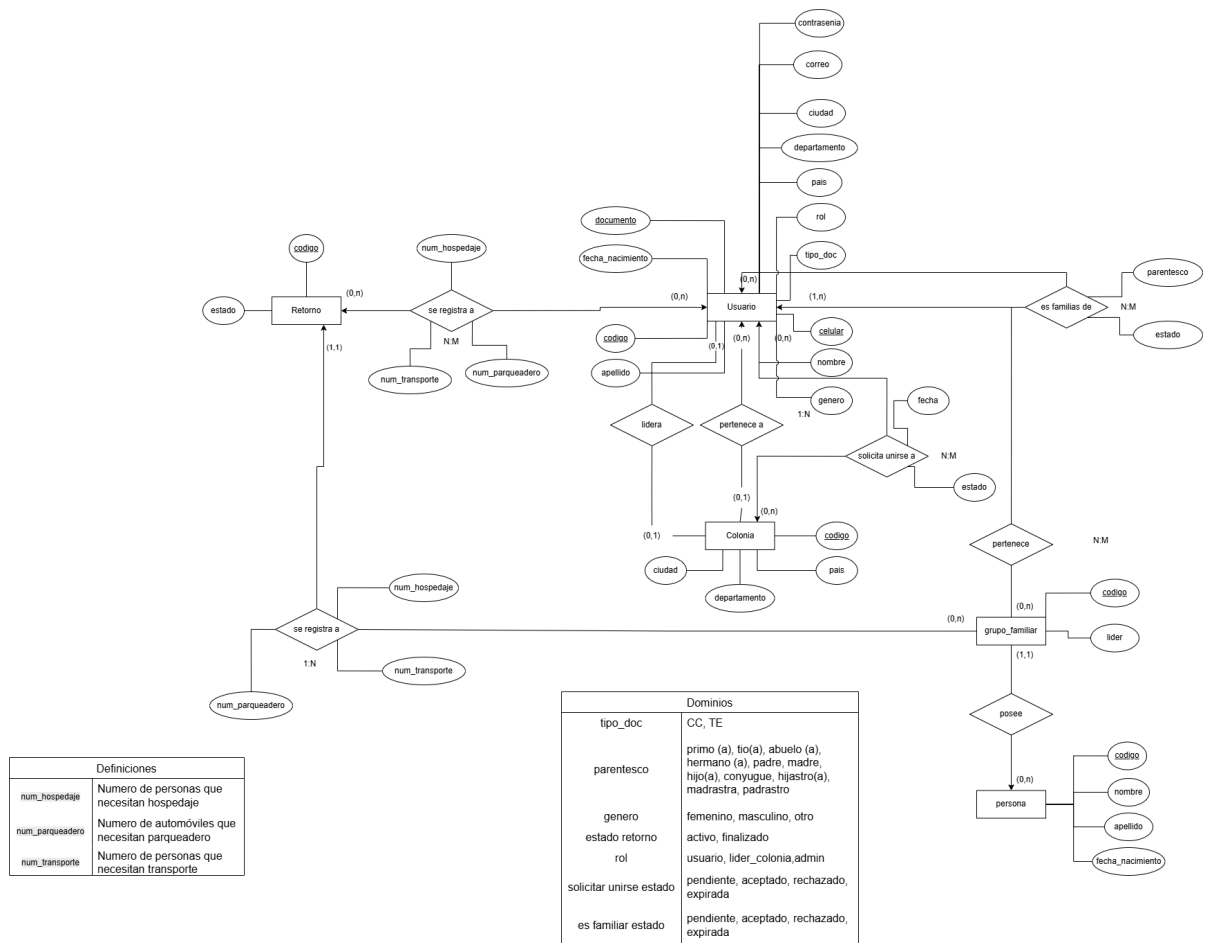


Figura 5. Diagrama entidad relación

[Ver Diagrama](#)

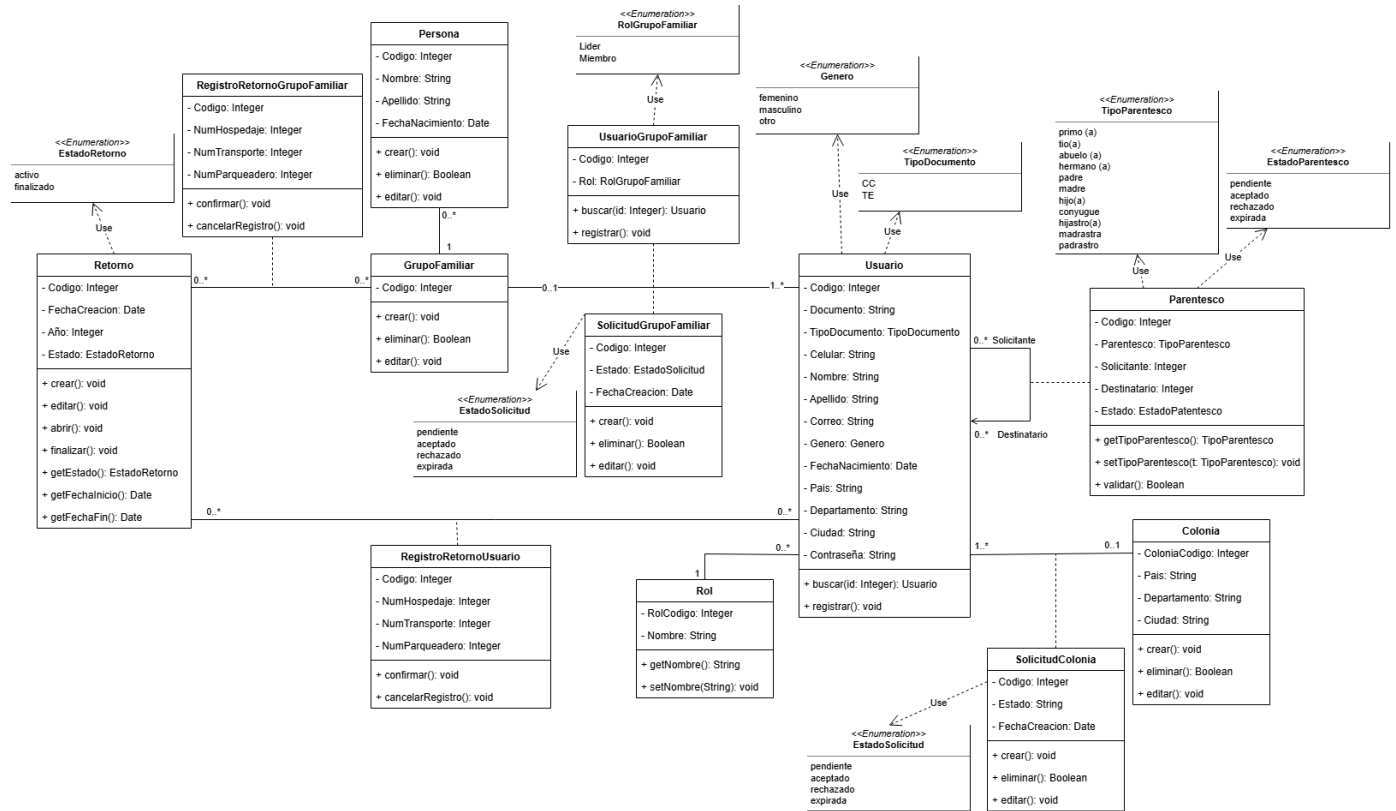


Figura 6. Diagrama de clases

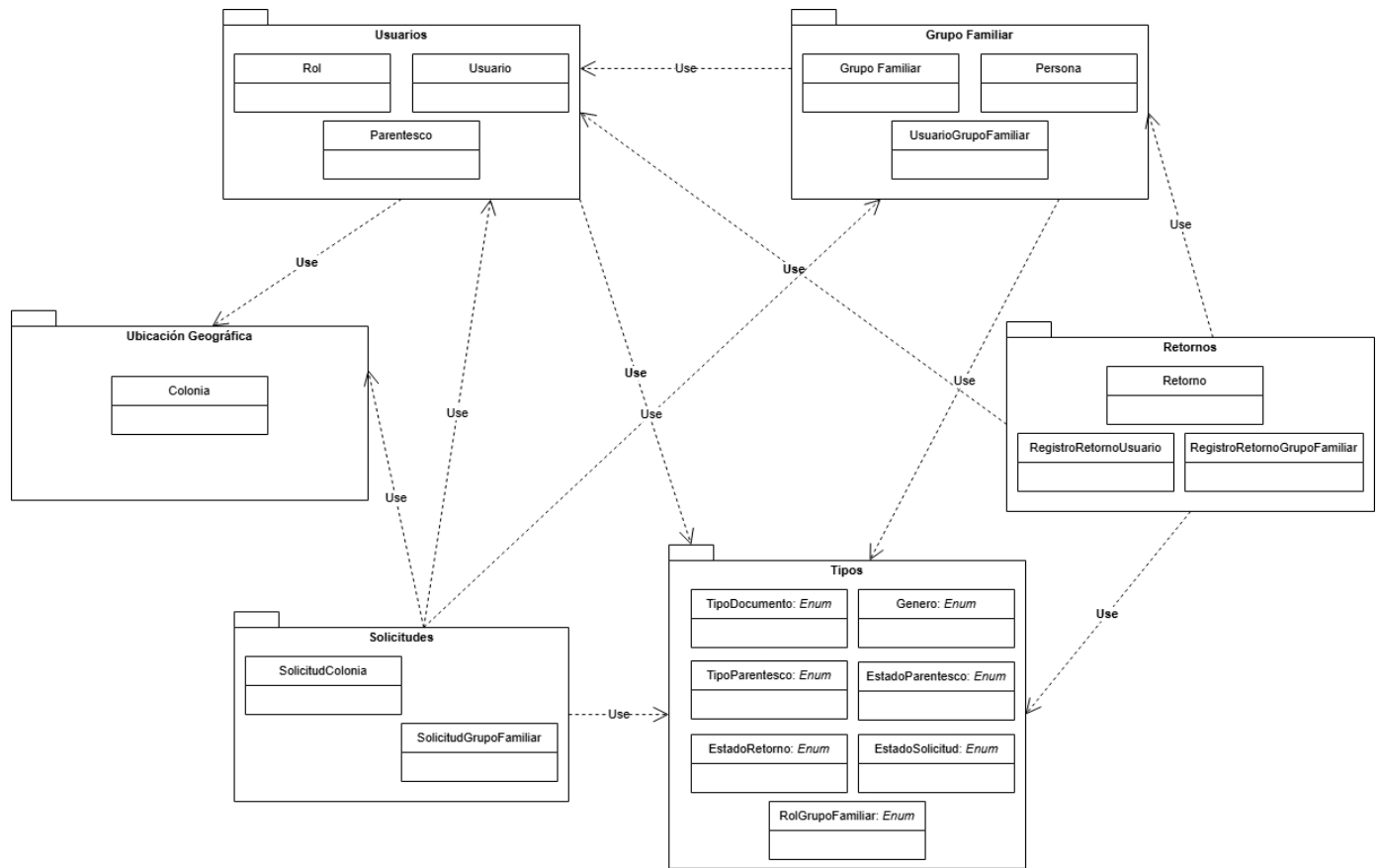


Figura 7. Diagrama de paquetes

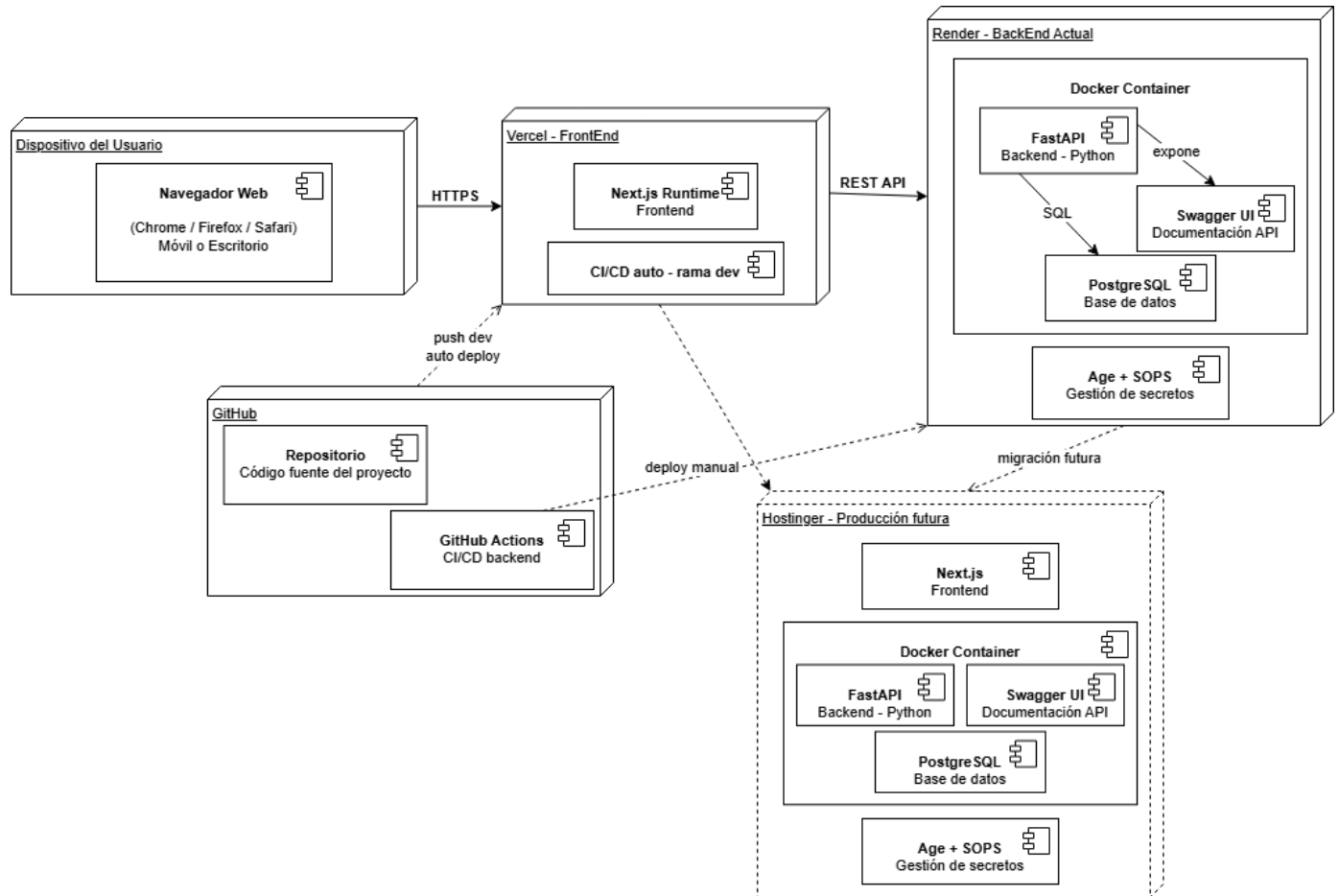


Figura 8. Diagrama de despliegue

Herramientas utilizadas

Jira

Jira es una plataforma de gestión ágil ampliamente utilizada en entornos SCRUM. Permite organizar y priorizar el product backlog, planificar los sprints, gestionar el sprint backlog y hacer seguimiento del progreso mediante tableros (Scrum/Kanban). Además, facilita la asignación de tareas, el control de incidencias y la generación de reportes como burndown charts.

Figma

Figma es una herramienta de diseño colaborativo basada en la nube utilizada para crear prototipos de interfaces de usuario (UI/UX). Permite diseñar de forma interactiva, compartir prototipos en tiempo real y obtener retroalimentación del equipo, lo que facilita la validación temprana de requisitos antes del desarrollo.

Notion

Notion es una plataforma integral de documentación y gestión del conocimiento. Se utiliza para estructurar historias de usuario, mantener especificaciones técnicas y centralizar información del proyecto. Su capacidad de enlazar contenido permite relacionar historias con prototipos, tareas y documentación relevante.

Slack

Slack es una herramienta de comunicación colaborativa que permite la interacción en tiempo real mediante canales, mensajes directos e integraciones con otras herramientas. En este caso, se complementa con el bot Geekbot, que automatiza encuestas diarias (daily check-ins) para monitorear el estado, avances y motivación del equipo de desarrollo.

Historias

Para el Sprint 2, el flujo de trabajo que se llevará a cabo involucra la terminación de las historias épicas de Gestión de Retornantes, Gestión de Colonias y se implementará también la Gestión de Reportes para cerrar el flujo desde la creación hasta las asistencias confirmadas, además, se incluye la Gestión de Multimedia, que añade el factor de valor característico del sprint puesto que se cumple uno de los requisitos iniciales del cliente, además de las gestiones iniciales completas.

En el primer link se encuentran en Notion, ya que permitió facilidades para incorporar los prototipos, también está el documento de Google Sheets para comodidad y orden, en especial para el test, y por último el acceso a Jira donde se definen y asignan.

Acceso a [Notion](#), a [Google Sheets](#), [Jira](#) y [Figma](#) para los prototipos.

Reunión Retrospectiva

Desarrollo

La técnica fue llevada a cabo de manera digital, utilizando una plantilla de la plataforma MIRO. Se asignó una franja de tiempo para que los miembros del equipo plasmarán sus pensamientos en cada uno de las secciones, posteriormente el equipo de trabajo participó en una reunión de meet para socializar los diferentes aspectos mencionados en cada una de las secciones, finalmente, apoyados por una herramienta de inteligencia artificial proporcionada por la plantilla se generó un listado de acciones para corregir y mejorar los diferentes aspectos mencionados durante la reunión.

Técnica

Para llevar a cabo la reunión retrospectiva se empleó la técnica 4L, este es un método estructurado utilizado en las retrospectivas de SCRUM para facilitar la reflexión del equipo sobre el sprint finalizado. Su nombre proviene de cuatro categorías que guían la discusión:

La técnica 4L organiza la retroalimentación del equipo en cuatro dimensiones clave, todas iniciadas con la letra “L” en inglés:

Liked (Lo que gustó): Aspectos positivos del sprint. Incluye prácticas, herramientas o dinámicas que funcionaron bien y deberían mantenerse.

Learned (Lo aprendido): Conocimientos adquiridos durante el sprint, tanto técnicos como de procesos. Puede incluir lecciones derivadas de errores o nuevos enfoques descubiertos.

Lacked (Lo que faltó): Elementos que el equipo considera que hicieron falta para mejorar el rendimiento, como recursos, información, comunicación o claridad en los requisitos.

Longed for (Lo que se desea): Ideas, mejoras o cambios que el equipo quisiera implementar en futuros sprints para optimizar el trabajo o el ambiente.

Liked

El equipo destacó principalmente aspectos positivos relacionados con la comunicación, el compromiso y el ambiente de trabajo. Se valoró que la comunicación fue clara, honesta y permitió llegar a acuerdos fácilmente, además de generar confianza para hacer preguntas.

También se resaltó la diligencia y responsabilidad en la realización de tareas, así como el cumplimiento de los objetivos, incluso cuando hubo retrasos. La revisión de código fue apreciada por su utilidad para detectar errores de manera temprana.

Finalmente, se reconoció un buen clima de equipo, caracterizado por el trato respetuoso, la amistad, la amenidad y el compromiso conjunto, junto con una actitud rigurosa frente al trabajo.

Learned

El equipo aprendió principalmente sobre la importancia de la comunicación y la organización del trabajo dentro del proyecto. Se destacó el valor de comunicarse directamente con los stakeholders y de mantener un flujo de trabajo

claro apoyado en reuniones efectivas..

Además, se fortaleció la comprensión sobre la gestión adecuada de recursos y la importancia de una coordinación eficiente para el desarrollo del proyecto.

Lacked

El equipo identificó principalmente oportunidades de mejora en la definición de requisitos y la coordinación del trabajo.

Se evidenció la necesidad de mejorar la gestión de cambios, para asegurar que toda la información sea compartida de manera oportuna y clara entre los miembros del equipo. También se señaló la importancia de involucrar más a los stakeholders desde etapas tempranas.

En cuanto a la gestión del desarrollo, se destacó la falta de una mejor definición de criterios de aceptación y de contratos e interfaces claras, lo cual impacta la fluidez del trabajo. Asimismo, se identificó la necesidad de mejorar la gestión de dependencias entre tareas y la planificación ante cambios, para optimizar tiempos y organización.

En síntesis, el equipo requiere mayor claridad, alineación y estructuración en los procesos para mejorar la eficiencia del desarrollo

Longed for

El equipo expresó principalmente la necesidad de mejorar la gestión del tiempo y la organización de las actividades. Se destacó el deseo de contar con mayor disponibilidad para reuniones y desarrollo de tareas, así como una mejor distribución del tiempo para cumplir con las fechas establecidas.

También se señaló la importancia de definir mejor las historias de usuario y los deadlines, especialmente en procesos como testing, para evitar ambigüedades.

Además, se manifestó la necesidad de mayor puntualidad y eficiencia en las reuniones, buscando que sean más ágiles y productivas.

En conjunto, el equipo anhela una mejor planificación, claridad en los requerimientos y optimización del tiempo de trabajo.

Acciones

La síntesis IA propuso 12 acciones para corregir o mejorar los aspectos mencionados en la sección anterior, de estos el grupo seleccionó las más importantes para el contexto del proyecto:

1. Implementar deadlines internos para aprobación de cambios antes de testing
2. Crear protocolo de comunicación de cambios en canal general del equipo
3. Establecer checklist de calidad de código y proceso de code review obligatorio
4. Implementar matriz de dependencias entre tareas en Jira
5. Definir acuerdo de puntualidad y duración máxima de reuniones
6. Establecer ceremonia de reconocimiento de fortalezas del equipo

Riesgos del proyecto y Gestión Sugerida

- **Técnico, Experiencia Limitada:** Debido a la falta de experiencia en Fast API, [Next.js](#) y CI/CD con GitHub Actions, existe una alta probabilidad de que haya retrasos. La forma de mitigar aquello es dedicando días del Sprint 0 y 1 a spikes técnicos donde habrá estudio activo con los cursos/tutoriales mencionados en el documento actual, y aprovechando la oportunidad de pair programming ya que tanto en back como en front, al menos un integrante conoce la tecnología nueva.
- **Seguridad, Datos Sensibles:** Crítico impacto por manejo de cédulas, teléfonos y fotos/videos personales. Un hackeo destruiría la confianza de la comunidad. Solución: JWT authentication, Row Level Security en Supabase (cada usuario solo ve su data) y logs de auditoría completos, siguiendo estándares GDPR-like.
- **Alcance, Cambio de Requisitos:** Alta probabilidad de cambios por feedback de presidentes de colonia durante validaciones cada 15 días. Se controla con historias de usuario flexibles, Definition of Done (no negociables) estricto y capacidad de pivotar prioridades en Sprint Review (al final) sin comprometer cronograma.
- **Operativo, Supabase Multimedia:** Media probabilidad de fallos con Supabase buckets para multimedia (límite 50MB/archivo, equipo sin experiencia). Prueba de concepto con fallback inmediato a r2 cloudflare y validaciones frontend que rechazan archivos grandes antes del upload.